

Smart Pest Management System - Core Implementation

This document contains the most important parts of the code for the Smart Pest Management System, including backend logic, AI model training, and key frontend components.

Table of Contents

- [backend/app.py](#)
 - [backend/train_improved.py](#)
 - [backend/train_transformer.py](#)
 - [src/App.jsx](#)
 - [src/components/LandingPage.jsx](#)
 - [src/components/UploadAnalyzer.jsx](#)
 - [src/components/AdminDashboard.jsx](#)
 - [src/components/WeatherWidget.jsx](#)
 - [src/components/Chatbot.jsx](#)
-

backend/app.py

```
import os
import io
import torch
import torch.nn as nn
from torchvision import models, transforms
# import tensorflow as tf # Removed for PyTorch migration
# from tensorflow.keras.models import load_model
# from tensorflow.keras.preprocessing import image
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS
from PIL import Image, UnidentifiedImageError
import pymongo
from pymongo import MongoClient
from bson.objectid import ObjectId
from datetime import datetime, timedelta
import datetime
from dotenv import load_dotenv
# Removed password hashing imports as per user requestv
from werkzeug.utils import secure_filename
import json
import csv
import re
import difflib # For local fuzzy matching
import zipfile
import tarfile
import shutil
import random
from split_dataset import split_dataset # Helper script
```

```

# Load environment variables
# Load environment variables correctly from backend/.env
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
load_dotenv(os.path.join(BASE_DIR, '.env'))

# Loaded via explicit logic above

# --- Flask App Initialization ---
app = Flask(__name__, static_folder="..../dist", static_url_path="/")
CORS(app) # Enable CORS for all routes

# Configuration
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_FOLDER = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'uploads')
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, 'profiles'), exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, 'medicines'), exist_ok=True)

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB max limit

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/uploads/<path:filename>')
def serve_uploads(filename):
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename)

@app.route('/api/admin/health', methods=['GET'])
def health_check():
    # Debug specific ID
    target_id = "Apple-Cedar-apple-rust"
    in_cache = target_id in DISEASE_DATA_DICT
    in_db = treatments_collection.find_one({"_id": target_id}) is not None if
treatments_collection is not None else False

    status = {
        "db_connected": client is not None,
        "treatments_count": treatments_collection.count_documents({}) if
treatments_collection is not None else 0,
        "disease_cache_size": len(DISEASE_DATA_DICT),
        "mongo_uri_masked": MONGO_URI.replace(MONGO_URI.split("@")[0].split("//")[1], "****")
    if "@" in MONGO_URI else "localhost",
        "debug_target": {
            "id": target_id,
            "in_cache": in_cache,

```

```

        "in_db": in_db,
        "cache_entry": str(DISEASE_DATA_DICT.get(target_id, "MISSING"))[:100]
    }
}
return jsonify(status), 200

@app.route("/")
def serve_index():
    return send_from_directory(app.static_folder, "index.html")

@app.errorhandler(404)
def not_found(e):
    # If the request is for an API endpoint, return JSON 404
    if request.path.startswith((' /auth', ' /admin', ' /medicines', ' /shops', ' /consultations',
    '/messages', ' /predict', ' /treatments', ' /subsidies', ' /forum')):
        return jsonify({"error": "Endpoint not found"}), 404

    # Support for React Client-Side Routing
    return send_from_directory(app.static_folder, "index.html")

# --- Database Setup (MongoDB) ---
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/")
DB_NAME = "pest_control_db"

... (truncated: showing first 100 lines of 3118) ...

```

backend/train_improved.py

```

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, WeightedRandomSampler
import os
import json
import time
import copy
import numpy as np

# Configuration
DATA_DIR = os.path.join(os.path.dirname(__file__), "data", "final_complete_dataset")
MODEL_SAVE_PATH = os.path.join(os.path.dirname(__file__), "model_efficientnet.pth")
INDICES_SAVE_PATH = os.path.join(os.path.dirname(__file__), "class_indices_improved.json")
LOG_PATH = os.path.join(os.path.dirname(__file__), "training_improved.log")

IMG_SIZE = (224, 224)
BATCH_SIZE = 32 # Optimized with AMP
EPOCHS_HEAD = 3 # Reduced from 5
EPOCHS_FINE = 7 # Reduced from 15

```

```

# Early Stopping Class
class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.0):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0

def get_class_weights(dataset):
    targets = torch.tensor(dataset.targets)
    class_sample_count = torch.tensor(
        [(targets == t).sum() for t in torch.unique(targets, sorted=True)])
    weight = 1. / class_sample_count.float()
    samples_weight = torch.tensor([weight[t] for t in targets])
    return samples_weight

def train_model(model, dataloaders, dataset_sizes, criterion, optimizer, scheduler,
num_epochs=25, device='cpu', dry_run=False):
    pass # Replaced by train_with_checkpoint logic below

def main():
    import argparse
    parser = argparse.ArgumentParser(description='Train Improved Model')
    parser.add_argument('--epochs', type=int, default=10, help='Total epochs (ignored if
using heads/fine strategy mostly)')
    parser.add_argument('--dry-run', action='store_true', help='Run a short training for
testing')
    args = parser.parse_args()

    global EPOCHS_HEAD, EPOCHS_FINE
    if args.dry_run:
        print("DRY RUN MODE ENABLED: Reduced epochs and batches")
        EPOCHS_HEAD = 1
        EPOCHS_FINE = 1

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # 1. Data Transforms (Advanced Augmentation)
    data_transforms = {

```

```

        'train': transforms.Compose([
            transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0)),
            transforms.RandomRotation(30),
            transforms.RandomHorizontalFlip(),
            transforms.RandomVerticalFlip(p=0.2),
            transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2), # Add
color jitter
            transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
        'validation': transforms.Compose([
            transforms.Resize((256, 256)), # Resize larger then crop
            transforms.CenterCrop(IMG_SIZE),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
        ]),
    }

# 2. Load Data
print(f"Loading data from {DATA_DIR}...")
image_datasets = {x: datasets.ImageFolder(os.path.join(DATA_DIR, x), data_transforms[x])
                  for x in ['train', 'validation']}

# 3. Handle Imbalance with WeightedRandomSampler
print("Computing class weights for resampling...")
samples_weight = get_class_weights(image_datasets['train'])
sampler = WeightedRandomSampler(samples_weight, len(samples_weight))

dataloaders = {

... (truncated: showing first 100 lines of 378) ...

```

backend/train_transformer.py

```

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, WeightedRandomSampler
import os
import json
import time
import copy
import numpy as np

# Configuration
DATA_DIR = os.path.join(os.path.dirname(__file__), "data", "final_complete_dataset")
MODEL_SAVE_PATH = os.path.join(os.path.dirname(__file__), "model_swin.pth")
INDICES_SAVE_PATH = os.path.join(os.path.dirname(__file__), "class_indices_improved.json")

```

```

LOG_PATH = os.path.join(os.path.dirname(__file__), "training_swin.log")

IMG_SIZE = (224, 224)
BATCH_SIZE = 32 # Optimized Batch Size
EPOCHS_HEAD = 3 # Reduced from 5
EPOCHS_FINE = 7 # Reduced from 15

# Early Stopping Class
class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.0):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0

def get_class_weights(dataset):
    targets = torch.tensor(dataset.targets)
    class_sample_count = torch.tensor(
        [(targets == t).sum() for t in torch.unique(targets, sorted=True)])
    weight = 1. / class_sample_count.float()
    samples_weight = torch.tensor([weight[t] for t in targets])
    return samples_weight

def train_model(model, dataloaders, dataset_sizes, criterion, optimizer, scheduler,
num_epochs=25, device='cpu', dry_run=False):
    pass # Replaced by train_with_checkpoint logic below

def main():
    import argparse
    parser = argparse.ArgumentParser(description='Train Swin Transformer')
    parser.add_argument('--epochs', type=int, default=10, help='Total epochs')
    parser.add_argument('--dry-run', action='store_true', help='Run a short training for
testing')
    args = parser.parse_args()

    global EPOCHS_HEAD, EPOCHS_FINE
    if args.dry_run:
        print("DRY RUN MODE ENABLED")
        EPOCHS_HEAD = 1
        EPOCHS_FINE = 1

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Data Transforms (Same sturdy augmentation)
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0)),
        transforms.RandomRotation(30),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(p=0.2),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'validation': transforms.Compose([
        transforms.Resize((256, 256)),
        transforms.CenterCrop(IMG_SIZE),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

print(f"Loading data from {DATA_DIR}...")
image_datasets = {x: datasets.ImageFolder(os.path.join(DATA_DIR, x), data_transforms[x])
                  for x in ['train', 'validation']}

print("Computing class weights...")
samples_weight = get_class_weights(image_datasets['train'])
sampler = WeightedRandomSampler(samples_weight, len(samples_weight))

dataloaders = {
    'train': DataLoader(image_datasets['train'], batch_size=BATCH_SIZE, sampler=sampler,
num_workers=4, pin_memory=True),
    'validation': DataLoader(image_datasets['validation'], batch_size=BATCH_SIZE,
shuffle=False, num_workers=4, pin_memory=True)

... (truncated: showing first 100 lines of 382) ...

```

src/App.jsx

```

import { useState, useEffect } from 'react'
import { Leaf, Sprout, TreePine } from 'lucide-react';
import Header from './components/Header'
import Hero from './components/Hero'
import LandingPage from './components/LandingPage'
import UploadAnalyzer from './components/UploadAnalyzer'
import ResultsDisplay from './components/ResultsDisplay'
import SeasonalAdvice from './components/SeasonalAdvice'

```

```

import ShopList from './components/ShopList'
import Login from './components/Login'
import AdminDashboard from './components/AdminDashboard'
import SubsidyList from './components/SubsidyList'
import Forum from './components/Forum'
import MedicineProposal from './components/MedicineProposal'
import ExpertDashboard from './components/ExpertDashboard'
import Profile from './components/Profile'
import Messaging from './components/Messaging'
import Feedback from './components/Feedback'
import History from './components/History'
import Chatbot from './components/Chatbot'
import UserAnnouncements from './components/UserAnnouncements'
import Consultations from './components/Consultations'

const isProfileComplete = (user) => {
  if (!user) return false;
  return user.name && user.address && user.phone && user.pincodes;
};

function App() {
  const [user, setUser] = useState(null)
  const [analysisResult, setAnalysisResult] = useState(null)
  const [view, setView] = useState('landing') // 'landing' | 'login'
  const [currentPage, setCurrentPage] = useState('home'); // 'home' | 'subsidies' | 'forum'
  | 'proposal' | 'profile' | 'messaging' | 'shops' | 'consultations'
  const [expertTab, setExpertTab] = useState('consultations');
  const [adminTab, setAdminTab] = useState('users');

  useEffect(() => {
    // Check session storage for persisted login
    const storedUser = sessionStorage.getItem('pest_user')
    if (storedUser) {
      setUser(JSON.parse(storedUser))
    }
  }, [])

  const handleLogin = (userData) => {
    setUser(userData)
    sessionStorage.setItem('pest_user', JSON.stringify(userData))
  }

  const handleLogout = () => {
    setUser(null)
    setAnalysisResult(null)
    sessionStorage.removeItem('pest_user')
  }

  const handleAnalyze = (result) => {
    setAnalysisResult(result)
    window.scrollTo({ top: 0, behavior: 'smooth' })
  }
}

```



```

const handleReset = () => {
  setAnalysisResult(null)
  window.scrollTo({ top: 0, behavior: 'smooth' })
}

if (!user) {
  if (view === 'landing') {
    return <LoginPage onNavigateToLogin={() => setView('login')} />
  }
  return <Login onLogin={handleLogin} onBack={() => setView('landing')} />
}

// Admin View
if (user.role === 'admin') {
  return (
    <div className="min-h-screen bg-slate-50 font-sans">
      <Header
        user={user}
        onLogout={handleLogout}
        adminView={true}
        onNavigate={setAdminTab}
        currentView={adminTab}
      />
      <AdminDashboard
        user={user}
        onLogout={handleLogout}
        activeTab={adminTab}
        setActiveTab={setAdminTab}
      />
      <Chatbot />
    </div>
  )
}

// Expert View

if (user.role === 'expert') {
  return (
    <div className="min-h-screen bg-slate-50 font-sans">

```

... (truncated: showing first 100 lines of 176) ...

src/components/LandingPage.jsx

```

import React, { useState } from 'react';
import { Sprout, Sun, CloudRain, Thermometer, User, ArrowRight, Activity, Play, X, Zap,
Cloud, Award } from 'lucide-react';

```

```

import WeatherWidget from './WeatherWidget';

export default function LandingPage({ onNavigateToLogin }) {
  const [showDemo, setShowDemo] = useState(false);

  return (
    <div className="min-h-screen bg-amber-50/40 font-sans overflow-hidden flex flex-col
relative selection:bg-amber-200 selection:text-amber-900">

      {/* Immersive Breathtaking Background for Vibrant Earth Theme */}
      <div className="absolute top-[-10%] left-[-10%] w-[40rem] h-[40rem] bg-orange-
300/30 rounded-full mix-blend-multiply filter blur-[100px] pointer-events-none animate-
pulse-slow"></div>
      <div className="absolute top-[20%] right-[-10%] w-[35rem] h-[35rem] bg-emerald-
300/30 rounded-full mix-blend-multiply filter blur-[100px] pointer-events-none animate-
pulse-slow" style={{ animationDelay: '2s' }}></div>
      <div className="absolute bottom-[-20%] left-[20%] w-[45rem] h-[45rem] bg-amber-
300/30 rounded-full mix-blend-multiply filter blur-[100px] pointer-events-none animate-
pulse-slow" style={{ animationDelay: '4s' }}></div>

      {/* Container mapping to max-width to keep content readable but background full
*/}
      <div className="flex-grow flex flex-col max-w-[90rem] mx-auto w-full px-6 py-6
lg:px-16 lg:py-10">

        {/* Navbar */}
        <nav className="relative z-10 flex justify-between items-center mb-8 lg:mb-
12">

          <div className="flex items-center gap-3 group cursor-pointer">
            <div className="bg-emerald-500 rounded-xl p-2.5 shadow-lg shadow-
emerald-500/20 group-hover:scale-105 transition-transform">
              <Sprout className="w-6 h-6 text-white" />
            </div>
            <span className="text-2xl font-bold text-slate-800 tracking-tight">
              Smart Pest <span className="text-emerald-500 font-semibold ml-
1">Management System</span>
            </span>
          </div>
          <button
            onClick={onNavigateToLogin}
            className="flex items-center gap-2 bg-white hover:bg-slate-50 border
border-slate-200 text-slate-700 px-6 py-2.5 rounded-full text-sm font-bold transition-all
shadow-sm hover:shadow hover:-translate-y-0.5 active:translate-y-0"
          >
            <User className="w-4 h-4 text-emerald-600" />
            Login / Register
          </button>
        </nav>

        {/* Main Two-Column Content */}
        <main className="relative z-10 grid grid-cols-1 lg:grid-cols-12 gap-16
lg:gap-8 flex-grow items-start pt-4 lg:pt-8">

```

```

        {/* Left Column */}
        <div className="col-span-1 lg:col-span-5 flex flex-col justify-start
space-y-10 animate-fade-in-up pr-4">
            <div>
                <div className="inline-flex items-center gap-2 px-5 py-2.5
rounded-full bg-emerald-100/80 border border-emerald-200 text-emerald-700 text-xs font-bold
tracking-widest uppercase mb-8 backdrop-blur-sm shadow-sm hover:scale-105 transition-
transform cursor-default">
                    <span className="w-2 h-2 rounded-full bg-emerald-500
animate-pulse"></span>
                    AI-Powered Protection
                </div>
                <h1 className="text-6xl lg:text-[5.5rem] font-black text-slate-
800 leading-[1.05] tracking-tighter drop-shadow-sm">
                    Start Farming <br />
                    <span className="text-transparent bg-clip-text bg-gradient-
to-r from-emerald-500 to-teal-400">
                        Smarter, Not <br className="hidden sm:block" /> Harder
                    </span>
                </h1>
            </div>

            <p className="text-slate-600 text-lg max-w-lg leading-relaxed font-
medium">
                Protect your crops with our advanced AI diagnosis system.
                Identify pests instantly, get expert treatment plans, and maximize your yield like never
                before.
            </p>

            <div className="flex flex-col sm:flex-row gap-5 pt-2">
                <button
                    onClick={onNavigateToLogin}
                    className="bg-gradient-to-r from-teal-500 to-emerald-500
text-white px-8 py-4 rounded-2xl font-bold text-base flex items-center justify-center gap-3
hover:from-teal-600 hover:to-emerald-600 transition-all shadow-xl shadow-emerald-500/25
hover:-translate-y-1 hover:shadow-2xl hover:shadow-emerald-500/30 group"
                >
                    Get Started Free
                    <ArrowRight className="w-5 h-5 group-hover:translate-x-1
transition-transform" />
                </button>
                <button
                    onClick={() => setShowDemo(true)}
                    className="bg-white/80 backdrop-blur-sm text-slate-700 px-8
py-4 rounded-2xl font-bold text-base flex items-center justify-center gap-3 border border-
slate-200 hover:bg-white transition-all shadow-sm hover:shadow hover:-translate-y-1 group"
                >
                    <div className="bg-teal-50 w-8 h-8 rounded-full flex items-
center justify-center group-hover:bg-teal-100 transition-colors">
                        <Play className="w-4 h-4 text-teal-600 fill-teal-600 ml-
0.5" />

```

```

        </div>
        View Demo
      </button>
    </div>
  </div>

  { /* Right Column: Cards Grid */ }
  <div className="col-span-1 lg:col-span-7 flex flex-col gap-6 relative z-
10 w-full animate-fade-in-up" style={{ animationDelay: '0.1s' }}>
    { /* Decorative background behind cards */ }
    <div className="absolute inset-0 bg-gradient-to-tr from-white/40 to-
white/10 rounded-[4rem] -z-10 blur-xl"></div>

    { /* Weather Widget prominent */ }
    <div className="bg-white/90 backdrop-blur-xl rounded-[2.5rem] p-6
lg:p-8 shadow-[0_8px_30px_rgb(0,0,0,0.06)] border border-slate-100 hover:-translate-y-1
transition-transform w-full flex flex-col sm:flex-row items-center gap-6 lg:gap-8">
      <div className="flex-1 px-2">
        <h3 className="text-sm font-bold tracking-widest text-
emerald-600 uppercase mb-4 flex items-center gap-2">
          <Thermometer className="w-5 h-5 text-orange-400 animate-
pulse" />

          Live Field Conditions
        </h3>
        <p className="text-slate-500 text-sm leading-relaxed mb-4">
          Get real-time, hyper-local weather insights integrated
directly into your diagnostic workflow. Optimize your planting and spraying schedules with
precision accuracy tailored to your exact location.
        </p>
      </div>
      <div className="flex-shrink-0 w-full sm:w-auto flex justify-
center">
        <WeatherWidget />
      </div>
    </div>
  </div>

... (truncated: showing first 100 lines of 195) ...

```

src/components/UploadAnalyzer.jsx

```

import React, { useState, useEffect } from 'react';
import { Upload, Search, Image as ImageIcon, Loader2, Info, ChevronDown, ChevronUp, Leaf, X,
Sparkles, Scan, Smartphone } from 'lucide-react';

export default function UploadAnalyzer({ onAnalyze, user }) {
  const [diseaseData, setDiseaseData] = useState({});
  const [diagnosisType, setDiagnosisType] = useState('disease'); // 'disease' | 'pest'
  const [activeTab, setActiveTab] = useState('upload'); // 'upload' | 'text'

```

```

const [isAnalyzing, setIsAnalyzing] = useState(false);
const [preview, setPreview] = useState(null);
const [imageFile, setImageFile] = useState(null);
const [textInput, setTextInput] = useState('');
const [cropName, setCropName] = useState('');
const [diseaseName, setDiseaseName] = useState('');
const [symptoms, setSymptoms] = useState('');
const [error, setError] = useState(null);

const SUPPORTED_DATA = {
  "Apple": ["Apple Scab", "Black Rot", "Cedar Apple Rust", "Healthy"],
  "Blueberry": ["Healthy"],
  "Cherry": ["Powdery Mildew", "Healthy"],
  "Grape": ["Black Rot", "Esca (Black Measles)", "Leaf Blight", "Healthy"],
  "Orange": ["Haunglongbing (Citrus Greening)"],
  "Peach": ["Bacterial Spot", "Healthy"],
  "Pepper": ["Bacterial Spot", "Healthy"],
  "Potato": ["Early Blight", "Late Blight", "Healthy"],
  "Raspberry": ["Healthy"],
  "Soybean": ["Healthy"],
  "Squash": ["Powdery Mildew"],
  "Strawberry": ["Leaf Scorch", "Healthy"],
  "Tomato": ["Bacterial Spot", "Early Blight", "Late Blight", "Leaf Mold", "Septoria
Leaf Spot", "Spider Mites", "Target Spot", "Mosaic Virus", "Yellow Leaf Curl Virus",
"Healthy"]
};

useEffect(() => {
  fetch('http://127.0.0.1:5000/treatments')
    .then(res => res.json())
    .then(data => {
      setDiseaseData(data);
    })
    .catch(err => console.error("Failed to load disease data:", err));
}, []);

const handleFileChange = (e) => {
  const file = e.target.files[0];
  if (file) {
    setImageFile(file);
    const url = URL.createObjectURL(file);
    setPreview(url);
    setError(null);
  }
};

const handleAnalyze = async () => {
  setIsAnalyzing(true);
  setError(null);

```

```

    try {
      let result = null;

      if (activeTab === 'upload') {
        console.log("Using Local Model...");
        if (!imageFile) throw new Error("Please upload an image for local
analysis.");

        // Validate Mandatory Fields - REMOVED per user request
        // if (diagnosisType === 'disease' && !cropName.trim()) throw new
Error("Please enter the Crop Name to ensure accurate diagnosis.");
        // if (diagnosisType === 'disease' && !symptoms.trim()) throw new
Error("Please enter the Symptoms to help identifying the problem.");

        const formData = new FormData();
        formData.append('image', imageFile);
        formData.append('crop_name', cropName);
        formData.append('disease_name', diseaseName);
        formData.append('symptoms', symptoms);

        if (user) {
          formData.append('user_id', user.id || user._id);
          formData.append('username', user.username);
        }

        const localResponse = await fetch('http://127.0.0.1:5000/predict', {
          method: 'POST',
          body: formData
        });

        if (!localResponse.ok) throw new Error("Local analysis failed. Check
server.");

        const localData = await localResponse.json();

        // Use backend-provided treatment info if available (Preferred)
        let info = localData.treatment;

        if (!info) {
          // Fallback: Enrich Local Data with CSV/DB Data (Legacy/Offline flow)
          const csvData = diseaseData.csv_data || {};
          const dbData = diseaseData.db_data || [];
          const detectedClass = localData.class;
          const baseInfo = csvData[detectedClass];

          info = baseInfo ? JSON.parse(JSON.stringify(baseInfo)) : {

... (truncated: showing first 100 lines of 462) ...

```

src/components/AdminDashboard.jsx

```

import React, { useEffect, useState } from 'react';
import { Search, History, Shield, Logout, Store, Pill, List, Trash2, Edit, ScanLine, User,
Flag, FileInput, CheckCircle, XCircle, Plus, GraduationCap, FlaskConical, Sprout, Home, Map
as MapIcon, MapPin, Phone, MessageSquare, Send, TreePine, AlertCircle, ArrowRight } from
' lucide-react';
import AdminSidebar from './admin/AdminSidebar';
import DashboardStats from './admin/DashboardStats';
import UserManagement from './admin/UserManagement';
import DatasetManager from './admin/DatasetManager';
import FeedbackAnalysis from './admin/FeedbackAnalysis';
import ShopForm from './ShopForm';
import MedicineForm from './MedicineForm';
import ResultsDisplay from './ResultsDisplay';
import UploadAnalyzer from './UploadAnalyzer';

import KnowledgeBase from './admin/KnowledgeBase';
import RetrainPanel from './admin/RetrainPanel';
import AdminAnnouncements from './admin/AdminAnnouncements';
import ForumManagement from './admin/ForumManagement';

import Messaging from './Messaging';
import UserAnalysis from './admin/UserAnalysis';

export default function AdminDashboard({ user, onLogout, activeTab, setActiveTab }) {
  const [logs, setLogs] = useState([]);
  const [shops, setShops] = useState([]);
  const [medicines, setMedicines] = useState([]);
  const [subsidies, setSubsidies] = useState([]);
  const [proposals, setProposals] = useState([]);
  const [users, setUsers] = useState([]);
  const [globalSearch, setGlobalSearch] = useState('');
  const [globalRole, setGlobalRole] = useState('all');
  const [analysisResult, setAnalysisResult] = useState(null);
  const [loading, setLoading] = useState(true);
  const [editingShop, setEditingShop] = useState(null);
  const [editingMedicine, setEditingMedicine] = useState(null);
  const [medicineFilter, setMedicineFilter] = useState('All');

  const [newSubsidy, setNewSubsidy] = useState({ title: '', description: '', link: '',
image: null });
  const [newExpert, setNewExpert] = useState({ username: '', password: '', name: '',
specialization: '', phone: '' });
  const [diseaseData, setDiseaseData] = useState(null);
  const [messageTarget, setMessageTarget] = useState(null);

  useEffect(() => {
    fetchData();
  }, [activeTab]);

  const fetchData = async () => {
    setLoading(true);

```

```

    await Promise.all([
      fetchHistory(),
      fetchShops(),
      fetchMedicines(),
      fetchUsers(),
      fetchSubsidies(),
      fetchProposals(),
      fetchTreatments()
    ]);
    setLoading(false);
  });

const fetchHistory = async () => {
  try {
    const response = await fetch('http://localhost:5000/admin/history');
    if (response.ok) setLogs(await response.json());
  } catch (error) { console.error("History fetch error", error); }
};

const fetchShops = async () => {
  try {
    const response = await fetch('http://localhost:5000/shops');
    if (response.ok) setShops(await response.json());
  } catch (error) { console.error("Shops fetch error", error); }
};

const fetchMedicines = async () => {
  try {
    const response = await fetch('http://localhost:5000/medicines');
    if (response.ok) setMedicines(await response.json());
  } catch (error) { console.error("Medicines fetch error", error); }
};

const fetchUsers = async () => {
  try {
    const response = await fetch('http://localhost:5000/admin/users');
    if (response.ok) setUsers(await response.json());
  } catch (error) { console.error("Users fetch error", error); }
};

const fetchSubsidies = async () => {
  try {
    const response = await fetch('http://localhost:5000/subsidies');
    if (response.ok) setSubsidies(await response.json());
  } catch (error) { console.error("Subsidies fetch error", error); }
};

const fetchProposals = async () => {
  try {
    const response = await fetch('http://localhost:5000/api/admin/proposals');
    if (response.ok) setProposals(await response.json());
  } catch (error) { console.error("Proposals fetch error", error); }
};

```



```
};
```

... (truncated: showing first 100 lines of 1110) ...

src/components/WeatherWidget.jsx

```
import React, { useState, useEffect } from 'react';
import { Cloud, Sun, CloudRain, Droplets, Wind, MapPin } from 'lucide-react';

export default function WeatherWidget() {
  const [weather, setWeather] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Default to New Delhi coordinates or detect user location
    // user location detection can be blocked, so we'll fallback to a generic
    agricultural region in India if needed
    // or just fetch for "detected" location.

    const fetchWeather = async (lat, lon) => {
      try {
        const response = await fetch(
          `https://api.open-meteo.com/v1/forecast?
latitude=${lat}&longitude=${lon}&current=temperature_2m,relative_humidity_2m,weather_code,wind
        );
        const data = await response.json();
        setWeather(data.current);
        setLoading(false);
      } catch (error) {
        console.error("Error fetching weather:", error);
        setLoading(false);
      }
    };

    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(
        (position) => {
          fetchWeather(position.coords.latitude, position.coords.longitude);
        },
        (_) => {
          console.log("Location denied, using default (Delhi)");
          fetchWeather(28.61, 77.20);
        }
      );
    } else {
      fetchWeather(28.61, 77.20);
    }
  }, []);
```

```

const getWeatherIcon = (code) => {
  if (code <= 3) return <Sun className="w-8 h-8 text-yellow-500 drop-shadow-sm" />;
  if (code <= 60) return <Cloud className="w-8 h-8 text-slate-100 drop-shadow-sm" />;
  return <CloudRain className="w-8 h-8 text-blue-200 drop-shadow-sm" />;
};

if (loading) return (
  <div className="bg-emerald-900/90 backdrop-blur-xl rounded-[2rem] p-6 animate-pulse w-72 h-32 border border-emerald-700/50 shadow-lg">
    <div className="h-4 bg-emerald-800 rounded w-1/2 mb-3"></div>
    <div className="h-10 bg-emerald-800 rounded w-1/3"></div>
  </div>
);

if (!weather) return (
  <div className="bg-emerald-900/90 backdrop-blur-xl border border-emerald-700/50 rounded-[2rem] p-6 text-white shadow-lg w-72">
    <p className="font-medium text-emerald-100">Weather data unavailable</p>
  </div>
);

return (
  <div className="bg-gradient-to-br from-emerald-800 to-teal-900 backdrop-blur-xl border border-emerald-700/50 rounded-[2rem] p-6 text-white shadow-xl shadow-emerald-900/20 w-72 transform hover:scale-105 transition-all duration-300 relative overflow-hidden group">
    { /* Inner Glow */ }
    <div className="absolute top-0 right-0 w-32 h-32 bg-emerald-400/10 rounded-full blur-2xl group-hover:bg-emerald-400/20 transition-colors pointer-events-none"></div>

    <div className="flex items-center justify-between mb-5 relative z-10">
      <div className="flex items-center gap-2 text-xs font-bold uppercase tracking-wider text-emerald-200">
        <MapPin className="w-4 h-4" />
        <span>Farm Conditions</span>
      </div>
      {getWeatherIcon(weather.weather_code)}
    </div>

    <div className="flex items-end gap-1 mb-5 relative z-10">
      <span className="text-5xl font-black tracking-tighter bg-clip-text text-transparent bg-gradient-to-b from-white to-emerald-50 drop-shadow-sm">
        {Math.round(weather.temperature_2m)}
      </span>
      <span className="text-2xl font-bold text-emerald-100/90 pb-1 tracking-tighter">°C</span>
    </div>

    <div className="grid grid-cols-2 gap-4 pt-4 border-t border-emerald-700/60 relative z-10">
      <div className="flex items-center gap-2">
        <Droplets className="w-4 h-4 text-emerald-300" />
        <span className="text-sm font-medium text-emerald-50">

```

```

{weather.relative_humidity_2m}% Humidity</span>
      </div>
      <div className="flex items-center gap-2">
        <Wind className="w-4 h-4 text-emerald-300" />
        <span className="text-sm font-medium text-emerald-50">
{weather.wind_speed_10m} km/h</span>
      </div>
    </div>
  </div>
);
}

```

src/components/Chatbot.jsx

```

import React, { useState, useRef, useEffect } from 'react';
import { MessageCircle, X, Send, Bot, Loader2 } from 'lucide-react';
import ReactMarkdown from 'react-markdown';

export default function Chatbot() {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState([
    { text: "Hi! I'm your Plant Doctor Assistant. How can I help you today?", sender:
'bot' }
  ]);
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const messagesEndRef = useRef(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages, isOpen]);

  const handleSend = async () => {
    if (!input.trim()) return;

    const userMessage = input.trim();
    setMessages(prev => [...prev, { text: userMessage, sender: 'user' }]);
    setInput("");
    setIsLoading(true);

    try {
      const response = await fetch('http://127.0.0.1:5000/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: userMessage })
      });
    }
  };

```

```

const data = await response.json();

if (data.reply) {
  setMessages(prev => [...prev, { text: data.reply, sender: 'bot' }]);
} else if (data.error) {
  setMessages(prev => [...prev, { text: "Error: " + data.error, sender: 'bot'
}]);
} else {
  setMessages(prev => [...prev, { text: "I'm not sure how to respond to
that.", sender: 'bot' }]);
}
} catch (error) {
  console.error("Chat error:", error);
  setMessages(prev => [...prev, { text: "Network error. Please check your
connection.", sender: 'bot' }]);
} finally {
  setIsLoading(false);
}
};

const handleKeyPress = (e) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    handleSend();
  }
};

return (
  <>
    { /* Floating Button */}
    <button
      onClick={() => setIsOpen(!isOpen)}
      className={`fixed bottom-6 right-6 p-4 rounded-full shadow-2xl transition-
all duration-300 z-50 flex items-center justify-center
${isOpen ? 'bg-red-500 rotate-90' : 'bg-nature-600 hover:bg-nature-700
hover:scale-110'} text-white`}
      aria-label="Toggle Chatbot"
    >
      {isOpen ? <X className="w-6 h-6" /> : <MessageCircle className="w-6 h-6" />}
    </button>

    { /* Chat Window */}
    <div className={`fixed bottom-24 right-6 w-80 sm:w-96 bg-white rounded-2xl
shadow-2xl border border-slate-100 z-40 overflow-hidden transition-all duration-300 origin-
bottom-right
${isOpen ? 'opacity-100 scale-100 translate-y-0' : 'opacity-0 scale-95 translate-y-
10 pointer-events-none'}`}>
      >
      { /* Header */}
      <div className="bg-gradient-to-r from-nature-600 to-nature-500 p-4 text-
white flex items-center gap-3">

```

```

    <div className="bg-white/20 p-2 rounded-full">
      <Bot className="w-5 h-5" />
    </div>
    <div>
      <h3 className="font-bold text-lg">Plant Doctor AI</h3>
      <p className="text-xs text-nature-100 flex items-center gap-1">
        <span className="w-2 h-2 bg-green-300 rounded-full animate-
pulse"></span>
        Online
      </p>
    </div>
  </div>

  { /* Messages */ }
  <div className="h-80 overflow-y-auto p-4 bg-slate-50 space-y-4">
    {messages.map((msg, idx) => (
      <div key={idx} className={`flex ${msg.sender === 'user' ? 'justify-
end' : 'justify-start'}`} >
        <div className={`max-w-[80%] p-3 rounded-2xl text-sm
${msg.sender === 'user'
? 'bg-nature-600 text-white rounded-tr-none'
: 'bg-white text-slate-700 shadow-sm border border-slate-100
rounded-tl-none'
} `}>
          <ReactMarkdown
            components={{
... (truncated: showing first 100 lines of 149) ...

```